

ECS401 Unit 11: Lab Progress Test: File I/O

FEEDBACK: Answers and Explanations

If you got any wrong you urgently need to do something to improve your understanding before it is too late.

1. Write a statement that links a variable called `haikuFile` to a file called `haiku.txt` to be used to read a Haiku poem in to a program from the file.

CORRECT ANSWER:

```
BufferedReader haikuFile =  
    new BufferedReader(new FileReader("haiku.txt"));
```

EXPLANATION:

This is a similar process to creating a Scanner, but instead we are connecting the named file to a `BufferedReader` variable. Within the `BufferedReader`, we are required to give a `FileReader`, which itself takes in a String containing the name of the file that we are wanting to read. The `FileReader` reads the actual characters from the file, and the `BufferedReader` buffers the data (i.e. gathers it together until the program is ready to use it.)

2. Write a line of code that reads a single line of text from a variable called `haikuFile` into a String variable called `line1`.

CORRECT ANSWER:

```
String line1 = haikuFile.readLine();
```

EXPLANATION:

This also works in a similar way to Scanner, where we just ask the `BufferedReader` to provide us with the first line in the file, and storing the value of that in a String variable called `line1`. As we called the `BufferedReader` variable `haikuFile` that is what we use to refer to the `readLine` method.

3. Write a line of code to close the file `haikuFile`.

CORRECT ANSWER:

```
haikuFile.close();
```

EXPLANATION:

We can close the file by calling the `close()` method. This releases the file from our program so that it can be accessed by other programs. Again as we called the `BufferedReader` variable `haikuFile` that is what we use to refer to the `readLine` method.

4. What needs to be added to the end of the header line of any method doing file input or output (including all methods in the call stack back to main that leads to it being called)?

CORRECT ANSWER:

```
... throws IOException
```

EXPLANATION:

Any method that deals with file I/O must throw an IOException. If there is an error within the program when trying to access a file, then this tells the program to deal with it by throwing an error and stopping execution. Ideally, we would design code that would avoid these errors in the first place – there are ways of doing this, but they are outside the scope of this module.

5. Write a statement that links a variable called haikuFile to a file called haiku.txt to be used to write a Haiku poem out from a program to the file.

CORRECT ANSWER:

```
PrintWriter haikuFile =  
    new PrintWriter(new FileWriter("haiku.txt"));
```

EXPLANATION:

The FileWriter has direct access to the file via the operating system and file system. The PrintWriter converts Strings into the appropriate sequence of characters to send to the file. These both work together to write to a file, the file name is given to the FileWriter when we create it.

6. Write a for loop that outputs the THREE lines of a Haiku to a file via a variable called haikuFile linked to the file for output. Assume the lines of the Haiku are stored in a String array variable called myHaiku.

CORRECT ANSWER:

```
for(int i = 0; i < 3; i++)  
{  
    haikuFile.println(myHaiku[i]);  
}
```

EXPLANATION:

The for loop will run three times, with the variable i will have the values 0, 1, and 2, respectively. This allows us to access the three elements in the array (remember that an array starts from 0). We can then print to the file using the PrintWriter, including the code that accesses the element as the input. As we called the PrintWriter variable haikuFile that is what we use to refer to the readLine method.

7. Explain briefly what would happen and why, if a program tried to write to a file that was still open for writing by another program if using the standard I/O mechanism as used in the lectures.

CORRECT ANSWER:

File systems allow programs to have a lock on a file while they are working on it. In general it is safe for lots of programs to read a file at once as they do not change it. However, writing to a file is more tricky and in general it is best that only one can write to a file at once. Therefore the first program to open a file for writing is given an interlock on it. This means it has exclusive access and nothing else can write to it once the interlock is given. A second program trying to write to a locked file would therefore raise an error (an alternative would be that it could wait for the interlock to be released when the file is closed by the one with the interlock). If another program does open the file for writing then it destroys the file contents already there of course. The file is also only guaranteed to be saved when the file is closed and the interlock released. If other programs are not allowed access till this happens then there is no problem of what happens depending on unpredictable timing of events in the different programs.

8. Which of the following fragments of code (a) - (e) below could be used to read in all the data from a file into an array of Student records. The file is stored in the format of one piece of data per line consisting of the student name followed by the mark they received on the next line, followed by the next name and next mark, and so on. The Student record is defined as follows

```
class Student
{
    int mark;
    String name;
}
```

The number of student records stored in the file is stored in variable ENTRIES. You can assume in each case there is appropriate other code to set up and close inputStream.

a)

```
for(int i = 0; i < ENTRIES; i++)
{
    String name = inputStream.readLine();
    int mark = Integer.parseInt(inputStream.readLine());
}
```

b)

```
Student [] student = new Student[ENTRIES];
String [] name = new String[ENTRIES];
int [] mark = new int[ENTRIES];

for(int i = 0; i < ENTRIES; i++)
{
    name[i] = inputStream.readLine();
    mark [i] = Integer.parseInt(inputStream.readLine());
}
```

c)

```
Student [] student = new Student[ENTRIES];

for(int i = 0; i< ENTRIES; i++)
{
    student[i].mark =
        Integer.parseInt(inputStream.readLine());
    student[i].name = inputStream.readLine();
}
```

d)

```
Student [] student = new Student[ENTRIES];

for(int i = 0; i< ENTRIES; i++)
{
    student[i].name = inputStream.readLine();
    student[i].mark =
        Integer.parseInt(inputStream.readLine());
}
```

e)

```
Student [] student = new Student[ENTRIES];

for(int i = 0; i< ENTRIES; i++)
{
    student.name[i] = inputStream.readLine();
    student.mark[i] =
        Integer.parseInt(inputStream.readLine());
}
```

CORRECT ANSWER:

d)

EXPLANATION:

- **d** is the correct code – it creates a new array of Student records, which are then filled correctly in the for loop.
- **a** is incorrect because it doesn't fill an array of records, instead just creating two new variables which are repeatedly redefined and the values stored lost at the end of the loop body executing as they are defined inside the loop body's block so have local scope.
- **b** is incorrect because even though it creates an array of records, it only stores the name and mark in two other arrays, which are not related at all to the array of records or the record itself at all.
- **c** is incorrect because the name and mark are the wrong way around in the for loop. The question says they are listed in the file with the name first, then mark. That means the name must be read in first then the mark, then another name and so on. The order they appear in the record is irrelevant to the order that those fields are filled. They can be filled in any order.
- **e** is incorrect because the square brackets to access array elements are on the name and mark, rather than the students array. This would only work if student was a record (not an array) as the field names of it are accessed.

Those fields would have to also be each an array as the next thing for each field is an array position is accessed. A record could be defined like that as containing fields that were arrays, but that is not how the record type Student is defined, nor how student is declared.